

Detección de objetos y segmentación

SPRINT 11 · VISIÓN POR COMPUTADOR · S11.3

Clasificar dice qué hay en la foto. Detectar dice qué y dónde. Segmentar dice qué píxeles. Y el reto exige saber cuándo no fiarse del modelo.

Tres tareas con imágenes

Tarea	Pregunta	Salida	Ejemplo TurisData
Clasificación	¿Qué hay en la foto?	una etiqueta	«esta foto es de una piscina»
Detección	¿Qué hay y dónde?	cajas (x1, y1, x2, y2) + clase + confianza	contar sombrillas en la playa
Segmentación	¿Qué píxeles son?	una máscara	superficie de arena libre

Cajas, IoU y NMS

```
def iou(a, b):                          # Ejercicio 1
    ancho = max(0, min(a[2], b[2]) - max(a[0], b[0]))
    alto  = max(0, min(a[3], b[3]) - max(a[1], b[1]))
    inter = ancho * alto
    return inter / (area(a) + area(b) - inter)
```

- IoU = área de intersección / área de unión (0 a 1). Acierta si IoU ≥ 0,5.
- NMS: el detector propone varias cajas casi iguales. Ordena por confianza, quédate con la primera y descarta las que se solapan con ella más del umbral; repite (nms(cajas, puntuaciones, umbral_iou=0.5), Ejercicio 2).

Métricas: precisión, recall, AP y mAP

4 sombrillas reales y 5 cajas (✔ verdadero positivo, ✘ falso positivo, con IoU ≥ 0,5):

Puesto	Confianza	¿Correcta?	Precisión acum.	Recall acum.
1	0,95	✔	1/1 = 1,00	1/4 = 0,25
2	0,90	✔	2/2 = 1,00	2/4 = 0,50
3	0,80	✘	2/3 = 0,67	2/4 = 0,50
4	0,70	✔	3/4 = 0,75	3/4 = 0,75
5	0,60	✘	3/5 = 0,60	3/4 = 0,75

AP = área bajo la curva precisión–recall, con la precisión «suavizada» de derecha a izquierda.

```
def average_precision(es_tp, n_reales):          # Ejercicio 3
    tp = np.cumsum(es_tp); fp = np.cumsum(1 - np.array(es_tp))
    precision, recall = tp / (tp + fp), tp / n_reales
    ...                                  # suavizar y sumar por aumento de recall
```

mAP@[.5:.9]: media del AP con umbrales de IoU 0,5…0,9 (Ejercicio 4).

Un detector clásico y la segmentación

- Escena sintética de la playa (128×128): arena, mar, sombrillas (círculos de colores, a veces solapados) y distractores.
- detector_color con OpenCV: umbral de color + contornos → cajas. evaluar_detector(...) da precisión, recall y AP.
- Segmentación: una máscara por píxel. Se mide con la IoU de máscaras o el Dice:

```
iou_mascara = (pred & real).sum() / (pred | real).sum()        # Ejercicio 5
dice = 2 * (pred & real).sum() / (pred.sum() + real.sum())     # 1.0 si ambas están vacías
```

Del cuaderno al reto: cuándo no fiarse

> Lo importante no es el acierto, sino saber cuándo no fiarse. Un modelo que acierta el 90 % pero no sabe cuándo falla es peligroso.

Pieza (cuaderno S11_04)	Función
Dataset por carpetas	contar_por_clase(ruta) (Ejercicio 1)
Transfer learning	modelo_transfer(base_preentrenada, n_clases) (Ejercicio 2)
Matriz de confusión	recall_clase, precision_clase desde cm (Ejercicio 3)
Análisis de errores	indices_errores(y_real, prob), de mayor a menor confianza (Ejercicio 4)
El modelo pide ayuda	umbral de confianza: cobertura y precisión sobre lo aceptado (Ejercicio 5)

El demostrador del reto

1. Preparar el conjunto (el sintético o uno propio con licencia): una carpeta por clase, sin espacios ni tildes, mínimo 30 imágenes por clase.
2. Transfer learning + referencia desde cero.
3. Evaluar en prueba y analizar al menos 6 fotos mal clasificadas, con una hipótesis de por qué.
4. Demostrador: una función que recibe una foto y devuelve clase, confianza y «revisar a mano» si la confianza es baja.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

Tipo	Recurso	Tiempo
CUADERNO · IoU, NMS, mAP, detector con OpenCV y segmentación	sprints/S11/notebooks/S11_03_deteccion_segmentacion	45 min
CUADERNO · Esqueleto del reto: dataset por carpetas y análisis de errores (arranque del reto)	sprints/S11/notebooks/S11_04_reto_vision_cliente	60 min
RETO · Preparar los datos (paso 1)	sprints/S11/guia/Reto_Sprint_11.md	185 min